UNIDAD DIDÁCTICA 7

DISTRIBUCIÓN DE APLICACIONES

MÓDULO PROFESIONAL:
DESARROLLO DE INTERFACES

ÍNDICE

RESUMEN INTRODUCTORIO 2
INTRODUCCIÓN 2
CASO INTRODUCTORIO 3
1. DISTRIBUCIÓN DE APLICACIONES 4
1.1 Componentes de una aplicación. Empaquetado 6
1.2 Tipos de empaquetamientos 6
1.2.1 Instaladores 7
1.2.2 Paquetes autoinstalables 8
2. CREACIÓN DE PAQUETE AUTOINSTALABLE 8
2.1 Herramientas para crear paquetes de instalación 9
2.1.1 Generar JAR 9
2.1.2 Launch4j 15
2.2 Personalización de la instalación 21
2.2.1 Inno Setup 22
2.2.2 Parámetros de la instalación 22
2.2.3 Logotipos, fondos, diálogos, botones, idioma, entre otros 25
2.2.4 Ficheros firmados digitalmente 27
2.3 Asistentes de instalación y desinstalación 28
2.4 Instalación, descarga y ejecución desde un servidor web 36
RESUMEN FINAL 42

RESUMEN INTRODUCTORIO

En esta unidad, abordamos uno de los aspectos clave dentro del ciclo de vida del software: la distribución e instalación de aplicaciones desarrolladas.

La distribución de aplicaciones consiste en preparar el software para su instalación en los dispositivos de los usuarios finales, adaptándolo según el sistema operativo y las necesidades del entorno de ejecución.

Exploraremos los diferentes componentes que conforman una aplicación y las alternativas para su empaquetado y distribución. También distinguiremos entre los distintos tipos de empaquetamientos, desde los instaladores tradicionales hasta los paquetes autoinstalables, que simplifican el proceso de instalación para los usuarios finales.

Además, analizaremos las herramientas más comunes para crear paquetes de instalación y cómo personalizarlos según los requisitos específicos del entorno de destino.

Finalmente, veremos cómo implementar instalaciones asistidas y desatendidas, así como la distribución de aplicaciones a través de servidores web.

INTRODUCCIÓN

Dentro del proceso de desarrollo, llega un momento en que es necesaria la planificación del despliegue o instalación del software en el usuario final.

En el caso de pequeñas aplicaciones, podemos pensar que la mejor forma de realizarlo es a través de paquetes autoinstalables ya que son fáciles de distribuir y además son fáciles de manejar por el usuario final acostumbrado en muchas ocasiones a estos interfaces.

En este punto, es necesario tener en cuenta el sistema operativo destino ya que dependerá de este cómo tengamos realizado el ejecutable y también cómo interactúa con el usuario final, su gestión, su mantenimiento y actualización y en última instancia su desinstalación.

CASO INTRODUCTORIO

En la empresa que has sido contratado como desarrollador junior, has desarrollado un pequeño software para realizar el registro de horario dentro de la empresa. Un pequeño software que una vez finalizado debes instalar en todos los equipos de los usuarios.

Tu jefe te encomienda la tarea de documentar e instalar la aplicación, pero antes se pregunta cuál es la mejor forma de realizarla teniendo en cuenta que la aplicación está realizada en Java.

Al final de esta unidad, tendrás los conocimientos para poder crear paquetes de instalación a partir de una aplicación realizada en Java, así como la creación de paquetes JAR propios de Java.

1. DISTRIBUCIÓN DE APLICACIONES

Una vez finalizada la aplicación, te planteas la instalación de la misma en todos los equipos de usuarios, ya que es esencial para el cumplimiento de la ley.

Una de las alternativas es mediante la creación de un JAR, la otra es mediante un paquete autoinstalable para Windows ya que el sistema operativo del usuario final es este.

El desarrollo de software tiene como objetivo lo que se denomina instalación o despliegue de la aplicación. Dentro de un posible modelo de desarrollo de software podemos tener las siguientes fases:

- **Análisis**: es la primera fase que habrá que llevar a cabo justo después de la propuesta de proyecto.

- **Diseño:** consiste a elaborar y tomar las decisiones adecuadas para el desarrollo correcto de la aplicación definida en la fase de análisis.

- **Desarrollo o codificación**: implementación del código que compondrá la aplicación o el sistema para desarrollar. Esta fase implica la implementación del que se ha diseñado y analizado a las fases anteriores. Cada módulo desarrollado tendrá que pasar las pruebas unitarias de manera individual.

- **Integración y pruebas:** la integración corresponde a coger todos los módulos ya finalizados e irlos juntando para formar la aplicación completa. Habrá que validar y verificar el funcionamiento correcto de todos los módulos por separado y de la aplicación una vez integrada. También habrá que revisar toda la documentación generada, tanto la técnica como la comercial o la de usuario.

- **Finalización y transferencia**: en esta fase se deberá tener en cuenta todo lo referente al empaquetamiento y la instalación de las aplicaciones, así como todo el que se ha ido decidiendo en cuanto a todas las fases anteriores.

  La fase de finalización y transferencia deberá tener en cuenta aspectos como la entrega del producto o aplicación desarrollada al cliente, la manera de instalarlo en los usuarios, su formación y los diferentes asesoramientos que habrá que hacer.

Además, el desarrollo de software es una tarea que se puede considerar en continua evolución a lo largo del tiempo. Ya sea una pequeña aplicación para un particular o para

una pequeña empresa donde será utilizada o un software a gran escala para muchos usuarios simultáneos, el producto desarrollado tendrá que seguir una instalación, una evolución y un mantenimiento.

Este mantenimiento se podrá dar por diferentes razones:

- Actualizaciones que los desarrolladores han hecho del software, por voluntad propia.
- Actualizaciones que ha reclamado el usuario.
- Actualizaciones necesarias por un cambio en el entorno.

ENLACE DE INTERÉS

Ya que vamos a tratar los temas de instalación y creación de instalables, es muy interesante ver conceptos que hoy en día están muy relacionados con desarrollo y despliegue de software.

VÍDEO DE INTERÉS

Comprueba las diferentes metodologías y arquitecturas de ciclo de vida del software.

1.1 Componentes de una aplicación. Empaquetado

Para comprender correctamente la manera de distribuir una aplicación informática hay que conocer antes cuáles son los elementos que la componen y las alternativas de empaquetamiento y distribución que hay.

Según el IEEE, el software o aplicaciones informáticas es aquel conjunto de programas de cómputo, procedimientos, reglas, documentación y datos asociados que forman parte de las operaciones de un sistema de computación.

Las aplicaciones informáticas o software están compuestas por muchos **componentes** del software que es necesario que sean ejecutados conjuntamente para que funcionen correctamente.

Asimismo, los componentes podrán estar preparados para ser utilizados (ejecutados) directamente por los usuarios finales, o se pueden crear para intercomunicarse con otros componentes del software.

Se puede considerar un componente del software todo recurso independiente, que haya sido desarrollado para una determinada tarea y que puede formar un entorno para ser ejecutado para resolver un procedimiento definido de manera única o en conjunto con otros componentes.

A partir de los componentes del software, hay que revisar otros componentes relacionados, que ayudarán a entender el procedimiento de la distribución, como:

- Bibliotecas ("librerías").
- Entorno a trabajo (framework o conjunto de componentes).
- Paquetes de software.
- Ensamblaje o assembly.

1.2 Tipos de empaquetamientos

Hay muchos tipos de paquetes y muchas maneras de empaquetar aplicaciones. Estas se pueden clasificar en función de los sistemas para los cuales se han desarrollado o las características de los paquetes. Hay paquetes para sistemas operativos concretos:

- Como pueden ser Windows (MSI), hay varias herramientas para la creación de paquetes del tipo MSI o EXE, pensados para sistemas operativos Windows. Algunas de estas herramientas son software de propiedad y otras son software libre.

- Linux (DEB, RPM). RPM, por ejemplo, es un formato de archivo usado en GNU/Linux. Es un tipo de paquete desarrollado originalmente para las distribuciones RedHat. Actualmente es un formato de paquetes utilizado por muchas distribuciones GNU/Linux, como SuSE o Fedora.
- MAC OS (PKG)
- Otros paquetes se podrán usar con diferentes sistemas operativos (PKG).

Por otro lado, tenemos los archivos JAR, Java Archive, que consiste en el archivo específico de Java que permite ejecutar aplicaciones. Como Java es un lenguaje multiplataforma, pude ser una opción fantástica para crear los instaladores.

1.2.1 Instaladores

La instalación consiste en el proceso por el que la aplicación o software es transferido, configurado y preparado para ser ejecutado en un determinado sistema operativo.

Incluso con formatos como los que hemos visto anteriormente, el formato JAR, todos los instaladores estarán ligados a un sistema operativo. En el caso del JAR, necesitamos previamente haber instalado como mínimo al menos el paquete JDK que nos permitiría ejecutar esos instaladores de tipo JAR.

Por regla general un instalador sigue el siguiente proceso:

1. Se verifica la compatibilidad de la versión que se está instalando con el sistema donde se pretende instalar. Aunque se pueden realizar muchas verificaciones, las más básicas corresponden a la versión del sistema operativo y de los requisitos de hardware.
2. Se verifica la integridad, es decir, que el instalador esté correcto.
3. Creación o uso de los recursos de directorios y ficheros necesarios.
4. Creación o uso de los usuarios necesarios, así como de los permisos del usuario que está usando el instalador.
5. Despliegue de todos los ficheros necesarios en los directorios creados.
6. Descarga, despliegue e instalación de librerías de terceros en el caso de necesidad. Por ejemplo, librerías de acceso a bases de datos, o incluso librerías de sistema específicas.
7. Configuración del sistema y/o aplicación en el caso de ser necesario
8. Configuración global de variables de sistemas o de entorno necesarias para la ejecución de la aplicación.

1.2.2 Paquetes autoinstalables

Aunque en el anterior punto, los instaladores, nos referíamos a ellos como aplicaciones totalmente autónomas, puede ser que sea necesaria la asistencia de un equipo técnico, sobre todo en el caso de aplicaciones específicas o de gestión empresarial.

Así pues, podemos distinguir dos tipos de instalaciones:

- Instalaciones realizadas por equipo técnico.
- Instalaciones realizadas por el propio usuario final.

Es en esta segunda acepción donde aparece el término paquetes autoinstalables, donde la interacción entre usuario e instalador es mínima, o suficientemente básica para que el paquete que realice la instalación tenga todos los recursos necesarios para poder realizar la instalación con éxito.

2. CREACIÓN DE PAQUETE AUTOINSTALABLE

Quieres realizar una primera muestra y prueba de generación de paquete autoinstalable y para ello has elegido Inno Setup para la creación de los paquetes autoinstalables, ya que los ordenadores clientes tienen sistemas operativos Windows.

Para crear un autoinstalable primero generarás un fichero JAR en Eclipse de tu código en lenguaje Java, luego realizarás un instalador .exe con Inno Setup, además le pondrás un logotipo y una fuente personalizada. Por último, lo distribuirás en la web de la empresa para que todos puedan descargarlo e instalarlo en sus ordenadores.

Uno de los mecanismos que hemos visto es la creación de paquetes autoinstalables que se puedan distribuir para su instalación final del software desarrollado. Un paquete autoinstalable podemos decir que tiene las siguientes partes:

- Un ejecutable relacionado con el sistema operativo en el que vayamos a realizar la instalación.
- Una documentación asociada a la instalación.
- Un sistema de ayudas para la instalación de paquetes necesarios para el correcto funcionamiento.

En este apartado veremos la parte más técnica en la realización de un paquete autoinstalable centrándonos en el lenguaje base que usamos a lo largo del módulo, Java. En el caso de otros lenguajes de programación centrados en aplicaciones desktop, el

proceso tendrá idénticos objetivos y donde difiera será en el proceso de creación de los paquetes.

La dificultad de tener que realizar un paquete de instalación para cada sistema operativo provoca que:

- El lenguaje de programación sea importante si nuestro objetivo es realizar ejecutables desktop multiplataforma. En ese punto Java, es desde hace mucho tiempo uno de los lenguajes más usados para este ámbito, aunque .NET ha hecho un cambio de filosofía en los últimos años.
- La arquitectura cliente/servidor ha pasado a ser la más usada en los últimos años, ya que evita esos problemas del sistema operativo destino y además mejora el mantenimiento.

ARTÍCULO DE INTERÉS

Comprende la importancia de saber cuándo una empresa necesita un modelo cliente-servidor o bien un modelo desktop:

2.1 Herramientas para crear paquetes de instalación

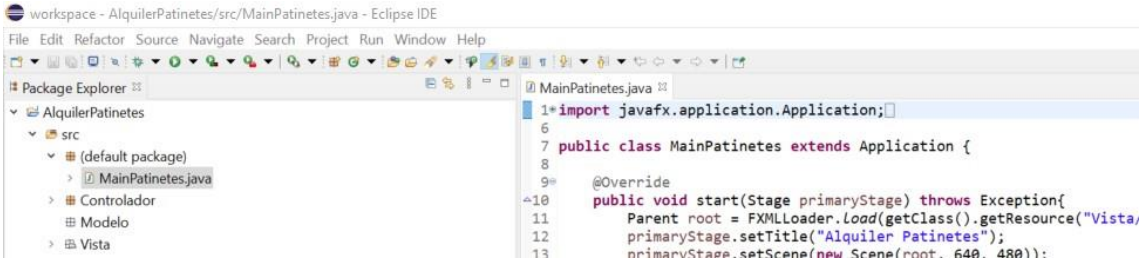
Al igual que veíamos anteriormente que el sistema operativo hace que las herramientas para la creación de paquetes de instalación sean dependientes del sistema operativo.

Partiendo de la idea base de que nuestro proyecto es un proyecto Java, vamos a plantear varias herramientas y formas para crear los paquetes de instalación.

2.1.1 Generar JAR

Una de las primeras opciones debe ser generar un JAR a través del propio software de desarrollo, en nuestro caso Eclipse.

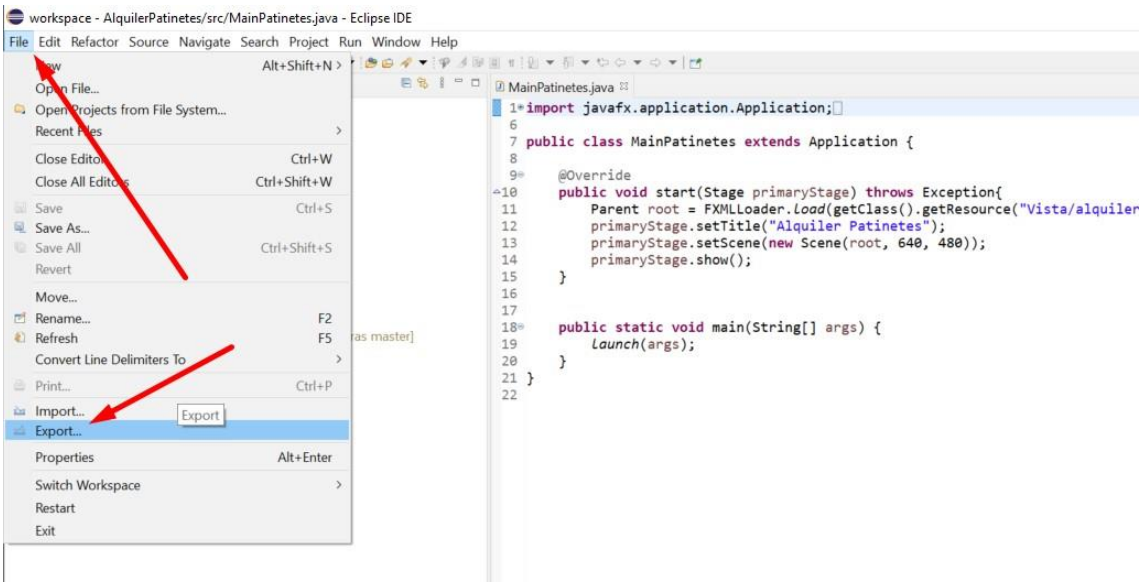
1. El primer paso será abrir el proyecto



Proyecto con Eclipse

Fuente: Elaboración propia

2. Desde el menú File, iremos a Export…



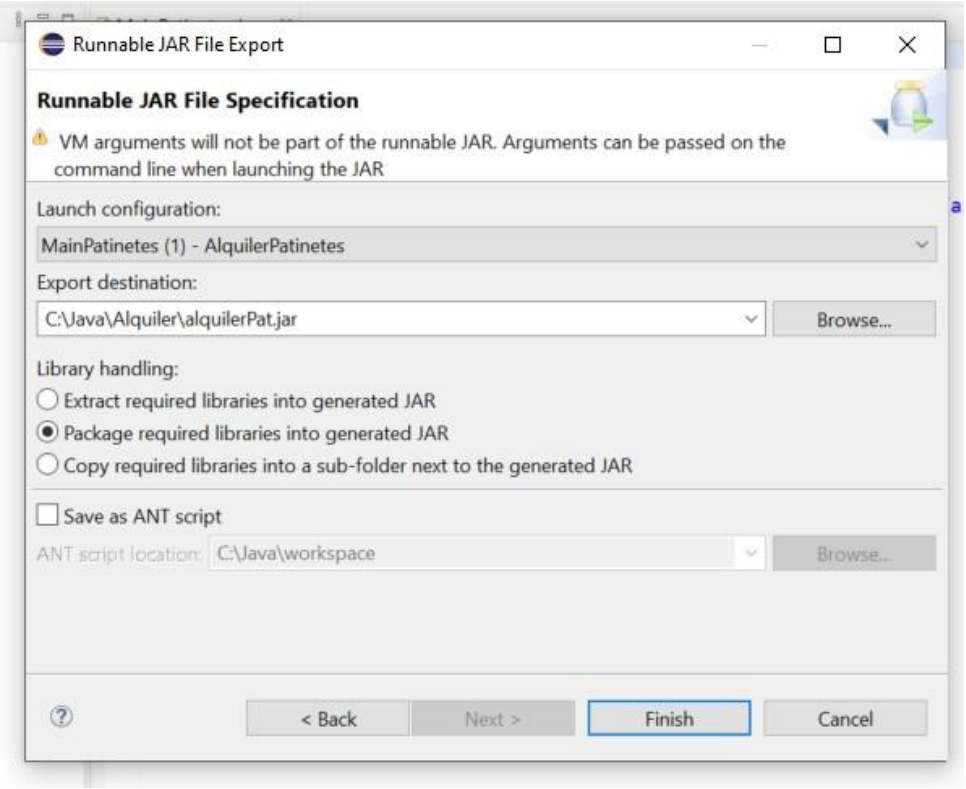
Exportar proyecto

Fuente: Elaboración propia

3. Elegiremos Runnable JAR file.

4. Dentro de las propiedades del JAR elegiremos:

   a. La configuración óptima de los parámetros de configuración.
   b. El nombre del fichero final.
   c. Empaquetar las librerías necesarias para la ejecución dentro del paquete JAR.



Parámetros para la exportación

Fuente: Elaboración propia

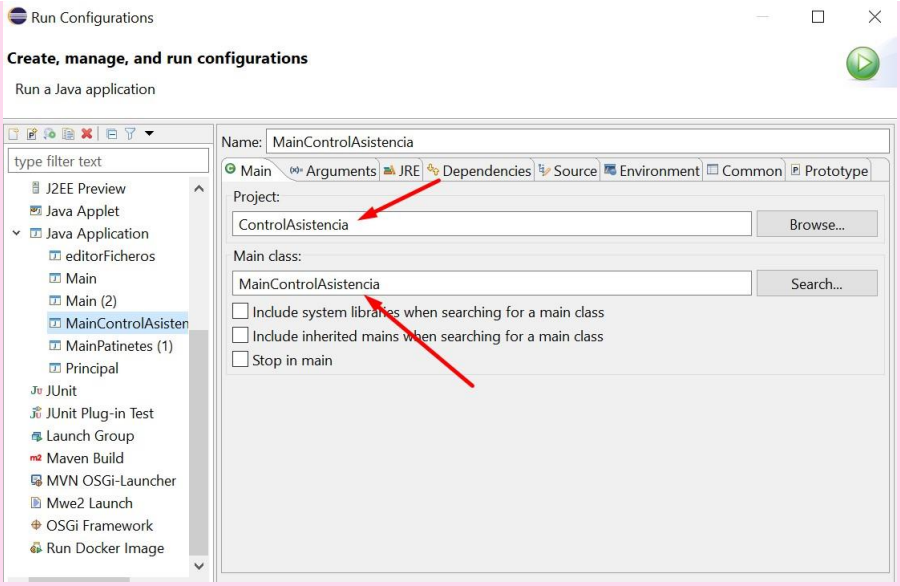
EJEMPLO PRÁCTICO

En el proceso de exportación de nuestra aplicación para el control de horas de los empleados queremos generar un primer paquete de exportación JAR. Teniendo en cuenta que la aplicación está realizada con las siguientes tecnologías:

- OpenJDK 21
- JFX 14
- JDBC
- MySQL

¿Cuál sería el paso a paso?

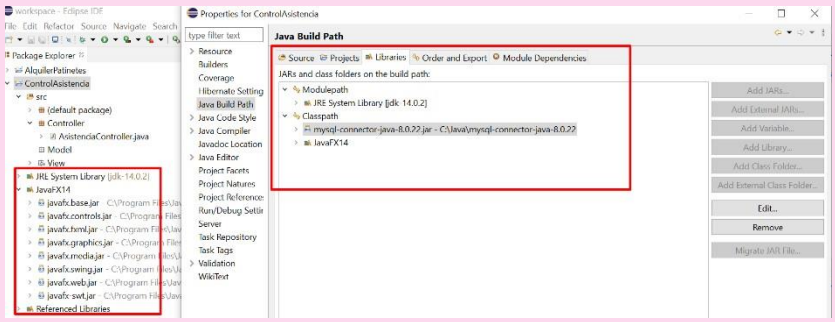
1) El primer paso es definir correctamente la clase con la que el loader de java debe arrancar, es decir, definir el main de nuestra aplicación.



Main de inicio

Fuente: Elaboración propia

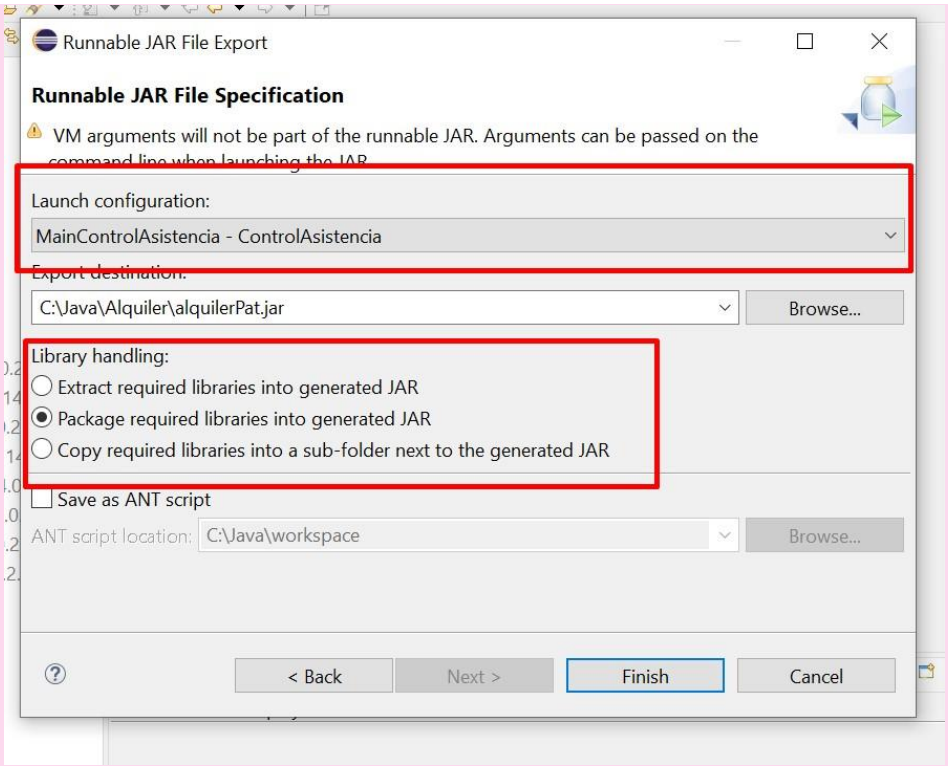
2) Una vez que se ha definido el main, debemos comprobar que en el proyecto se encuentran todas las librerías necesarias para incluirlas en nuestro paquete JAR.



Librerías del proyecto

Fuente: Elaboración propia

3) En ese punto ya podemos realizar el JAR de nuestro proyecto



Configuración de nuestro JAR

Fuente: Elaboración propia

EJEMPLO PRÁCTICO

Tu jefe solicita tu ayuda. Según comenta, tras haber recibido el fichero JAR ha intentado ejecutarlo y recibe el siguiente error: WARNING: Unsupported JavaFX configuration. ¿Cómo lo solucionarías?

1) El error se produce porque a veces JavaFX produce problemas cuando ejecutamos un jar para ello vamos a verificar que el error sucede. Pones el comando de ejecución: java -jar fichero.jar

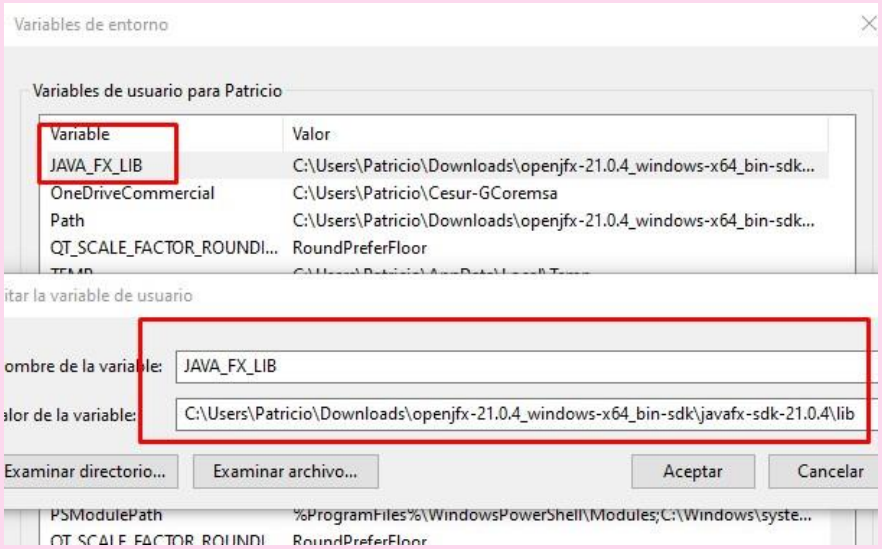
```
Seleccionar C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.19045.2913]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Patricio\Downloads\jar2>java -jar prueba.jar
nov 03, 2024 4:36:23 P. M. com.sun.javafx.application.PlatformImpl startup
WARNING: Unsupported JavaFX configuration: classes were loaded from 'unnamed module @45e286e2'
Graphics Device initialization failed for :  d3d, sw
Error initializing QuantumRenderer: no suitable pipeline found
java.lang.RuntimeException: java.lang.RuntimeException: Error initializing QuantumRenderer: no suitable pipeline found
        at com.sun.javafx.tk.quantum.QuantumRenderer.getInstance(QuantumRenderer.java:283)
        at com.sun.javafx.tk.quantum.QuantumToolkit.init(QuantumToolkit.java:253)
        at com.sun.javafx.tk.Toolkit.getToolkit(Toolkit.java:263)
        at com.sun.javafx.application.PlatformImpl.startup(PlatformImpl.java:290)
        at com.sun.javafx.application.PlatformImpl.startup(PlatformImpl.java:162)
        at com.sun.javafx.application.LauncherImpl.startToolkit(LauncherImpl.java:651)
        at com.sun.javafx.application.LauncherImpl.launchApplication1(LauncherImpl.java:671)
        at com.sun.javafx.application.LauncherImpl.lambda$launchApplication$2(LauncherImpl.java:196)
        at java.base/java.lang.Thread.run(Thread.java:1583)
Caused by: java.lang.RuntimeException: Error initializing QuantumRenderer: no suitable pipeline found
        at com.sun.javafx.tk.quantum.QuantumRenderer$PipelineRunnable.init(QuantumRenderer.java:95)
        at com.sun.javafx.tk.quantum.QuantumRenderer$PipelineRunnable.run(QuantumRenderer.java:125)
        ... 1 more
Exception in thread "main" java.lang.reflect.InvocationTargetException
        at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:118)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
        at org.eclipse.jdt.internal.jarinjarloader.JarRsrcLoader.main(JarRsrcLoader.java:63)
Caused by: java.lang.RuntimeException: No toolkit found
        at com.sun.javafx.tk.Toolkit.getToolkit(Toolkit.java:275)
        at com.sun.javafx.application.PlatformImpl.startup(PlatformImpl.java:290)
        at com.sun.javafx.application.PlatformImpl.startup(PlatformImpl.java:162)
```

Error fichero JAR

Fuente: Elaboración propia

2) Debes ejecutarlo especificando donde se encuentra Java FX, pero primero debes decirle al sistema dónde está. Para ello, estableces tu variable de entorno a la ruta: JAVA_FX_LIB – ruta de javafx.



Estableciendo variable de Entorno

Fuente: Elaboración propia

3) Una vez establecida, puedes ejecutar los .jar pasándole por parámetro la ruta de JavaFX: java --module-path %JAVA_FX_LIB% --add-modules javafx.controls,javafx.fxml -jar fichero.jar y ves que funciona perfectamente.

```
C:\Windows\System32\cmd.exe
        at com.sun.javafx.tk.quantum.QuantumRenderer.getInstance(QuantumRenderer.java:283)
        at com.sun.javafx.tk.quantum.QuantumToolkit.init(QuantumToolkit.java:253)
        at com.sun.javafx.tk.Toolkit.getToolkit(Toolkit.java:263)
        at com.sun.javafx.application.PlatformImpl.startup(PlatformImpl.java:290)
        at com.sun.javafx.application.PlatformImpl.startup(PlatformImpl.java:162)
        at com.sun.javafx.application.LauncherImpl.startToolkit(LauncherImpl.java:651)
        at com.sun.javafx.application.LauncherImpl.launchApplication1(LauncherImpl.java:671)
        at com.sun.javafx.application.LauncherImpl.lambda$launchApplication$2(LauncherImpl.java:196)
        at java.base/java.lang.Thread.run(Thread.java:1583)
Caused by: java.lang.RuntimeException: Error initializing QuantumRenderer: no suitable pipeline found
        at com.sun.javafx.tk.quantum.QuantumRenderer$PipelineRunnable.init(QuantumRenderer.java:95)
        at com.sun.javafx.tk.quantum.QuantumRenderer$PipelineRunnable.run(QuantumRenderer.java:125)
        ... 1 more
Exception in thread "main" java.lang.reflect.InvocationTargetException
        at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:118)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
        at org.eclipse.jdt.internal.jarinjarloader.JarRsrcLoader.main(JarRsrcLoader.java:63)
Caused by: java.lang.RuntimeException: No toolkit found
        at com.sun.javafx.tk.Toolkit.getToolkit(Toolkit.java:275)
        at com.sun.javafx.application.PlatformImpl.startup(PlatformImpl.java:290)
        at com.sun.javafx.application.PlatformImpl.startup(PlatformImpl.java:162)
        at com.sun.javafx.application.LauncherImpl.startToolkit(LauncherImpl.java:651)
        at com.sun.javafx.application.LauncherImpl.launchApplication1(LauncherImpl.java:671)
        at com.sun.javafx.application.LauncherImpl.lambda$launchApplication$2(LauncherImpl.java:196)
        at java.base/java.lang.Thread.run(Thread.java:1583)

C:\Users\Patricio\Downloads\jar2>java --module-path %JAVA_FX_LIB% --add-modules javafx.controls,javafx.fxml -jar prueba.
jar

C:\Users\Patricio\Downloads\jar2>
```

Ejecución JavaFX

Fuente: Elaboración propia

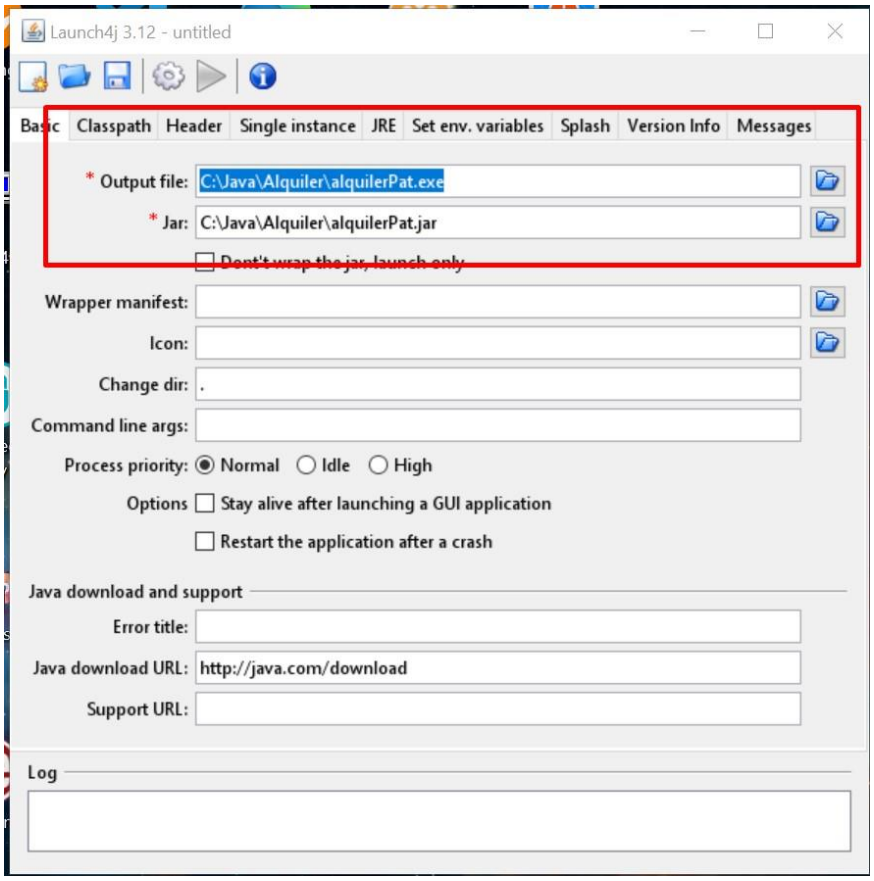
VÍDEO DE INTERÉS

Profundiza sobre la creación y el significado de creación de un ejecutable JAR.

2.1.2 Launch4j

Una herramienta muy madura pero que nos permite fácilmente realizar un ejecutable para entornos Windows es Launch4j.

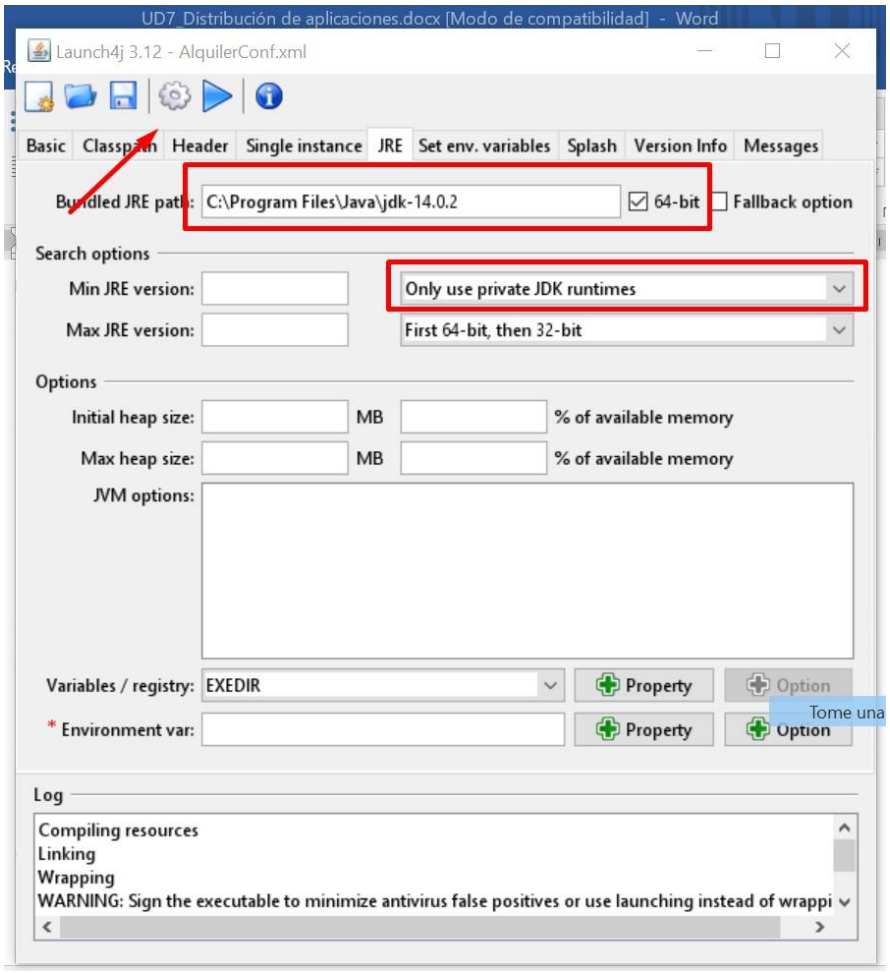
Una vez instalado simplemente elegiremos el fichero JAR creado a partir de nuestro proyecto y elegiremos el fichero ejecutable de salida.



Configuración de los directorios de generación de ejecutables

Fuente: Elaboración propia

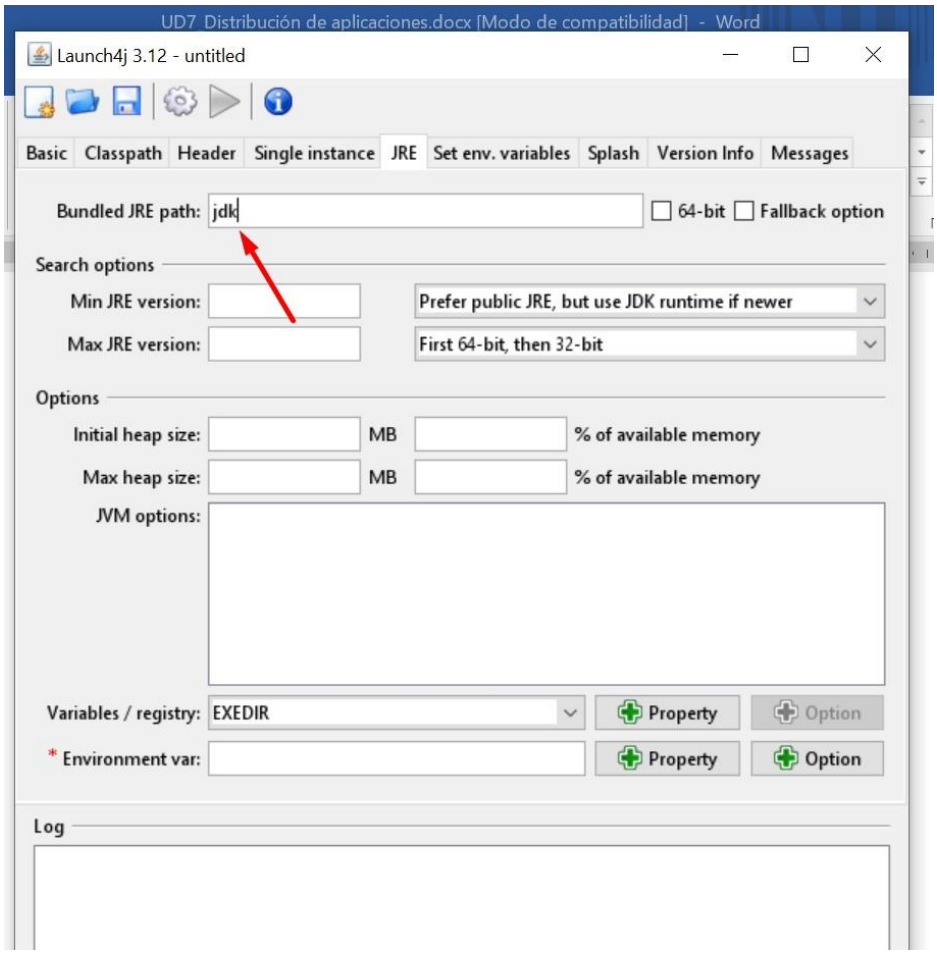
En el caso de haber usado librerías específicas como openJDK, es recomendable usar el path completo contra esas librerías.



Configuración del directorio de JRE

Fuente: Elaboración propia

Otra posible configuración que posteriormente la podemos distribuir en el autoinstalable, es guardar en la misma carpeta que tengamos el .JAR, la carpeta con el JRE y usar esas librerías para la configuración.



Path del directorio JRE

Fuente: Elaboración propia

ENLACE DE INTERÉS

Accede a la descarga y web oficial de launch4j.

EJEMPLO PRÁCTICO

En el proceso de exportación de nuestra aplicación para el control de horas de los empleados queremos realizar una primera prueba de ejecución de un ejecutable en un ordenador de un usuario final.

Para ello pretendemos crear un ejecutable para Windows mediante launch4j teniendo en cuenta que ya hemos generado el JAR con todas las librerías necesarias para el correcto funcionamiento.

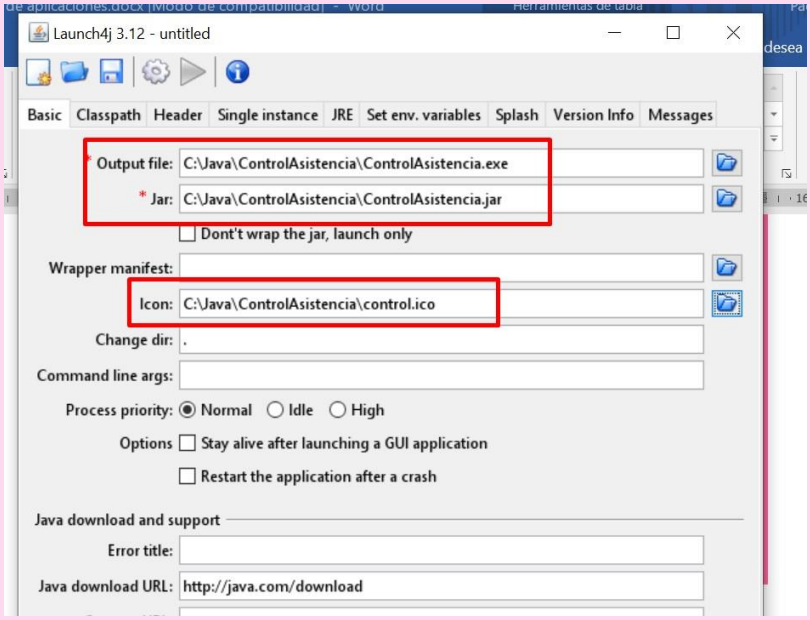
¿Cuál sería el paso a paso?

1) El primer paso sería incluir en la misma carpeta donde tenemos el JAR las librerías del OpenJDK para posteriormente evitar depender de las librerías instaladas en el equipo del usuario.

Librerías del JDK

Fuente: Elaboración propia

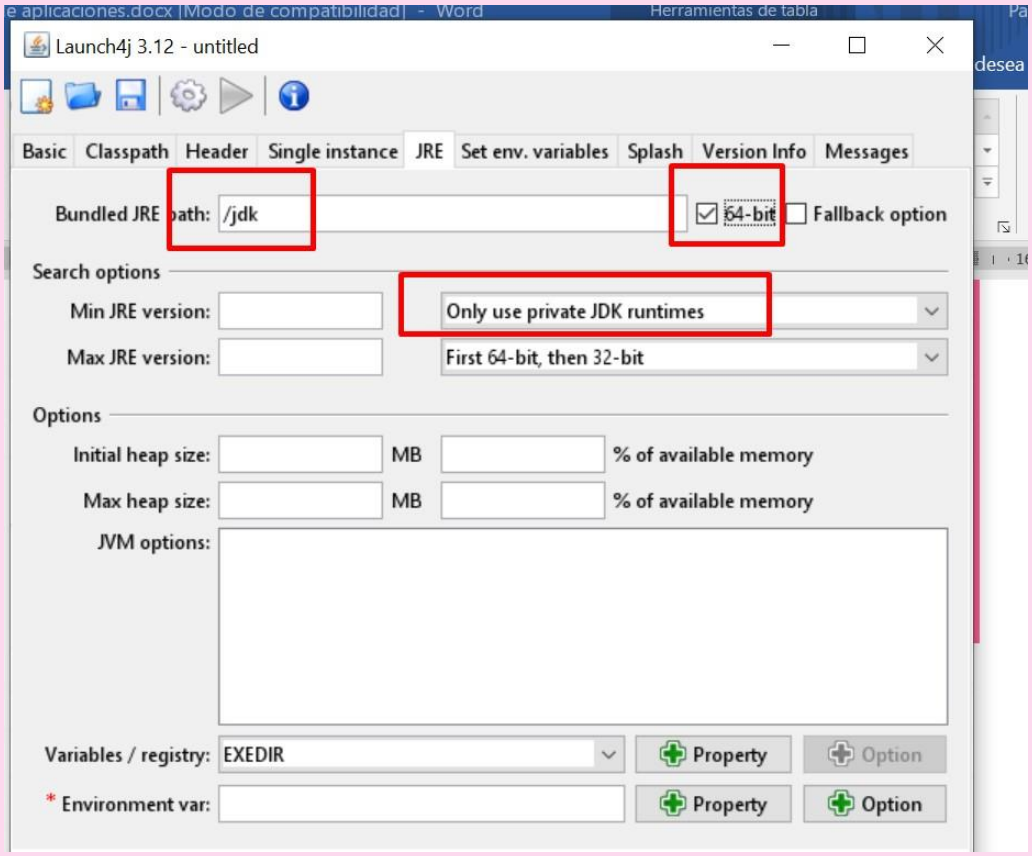
2) En ese punto ya podemos usar launch4j para realizar el ejecutable, añadiendo el JAR, y el icono del ejecutable.



Configuración básica del launch4j

Fuente: Elaboración propia

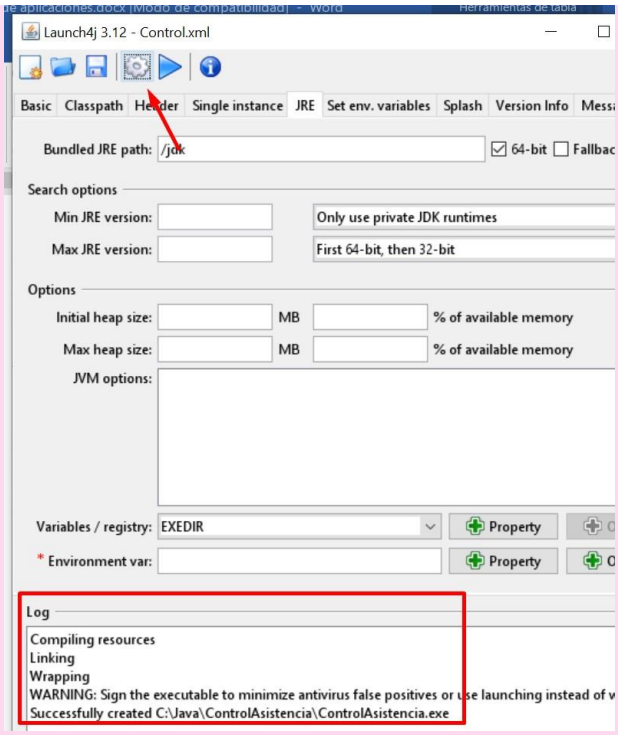
3) Marcamos las librerías internas que usaremos para la ejecución del JDK.



Configuración JDK

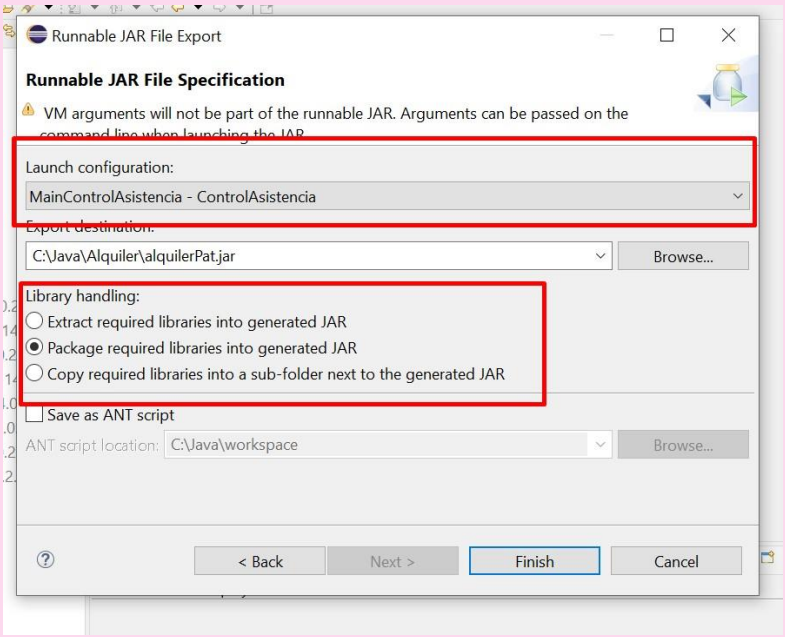
Fuente: Elaboración propia

4) Generaríamos el ejecutable.



Creación del ejecutable

Fuente: Elaboración propia



Configuración de nuestro JAR

Fuente: Elaboración propia

2.2 Personalización de la instalación

La personalización de la instalación de nuestra aplicación consiste en la adecuación de muchos parámetros para que en el equipo cliente funcione de la misma forma que en el equipo de desarrollo. Dependiendo de la complejidad de nuestro software, la

personalización puede necesitar desde unas pocas consideraciones hasta una instalación mucho más compleja y técnica:

- Personalización de imágenes, iconos, logos, etc.
- Personalización de directorios.
- Configuración de parámetros de bases de datos.
- Configuración de parámetros de servidor.

2.2.1 Inno Setup

Para la configuración de un paquete autoinstalable con el objetivo de ser usado en Windows, usaremos la aplicación Inno Setup, que además de Opensource, nos proporciona una enorme cantidad de posibilidades de configuraciones.

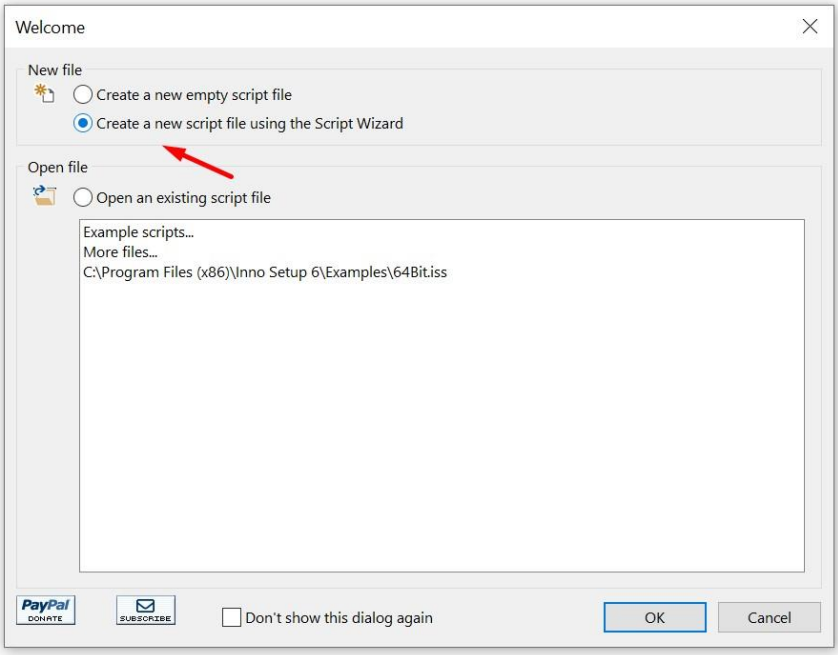
ENLACE DE INTERÉS

Aquí accederás a la descarga y web oficial de Inno Setup.

2.2.2 Parámetros de la instalación

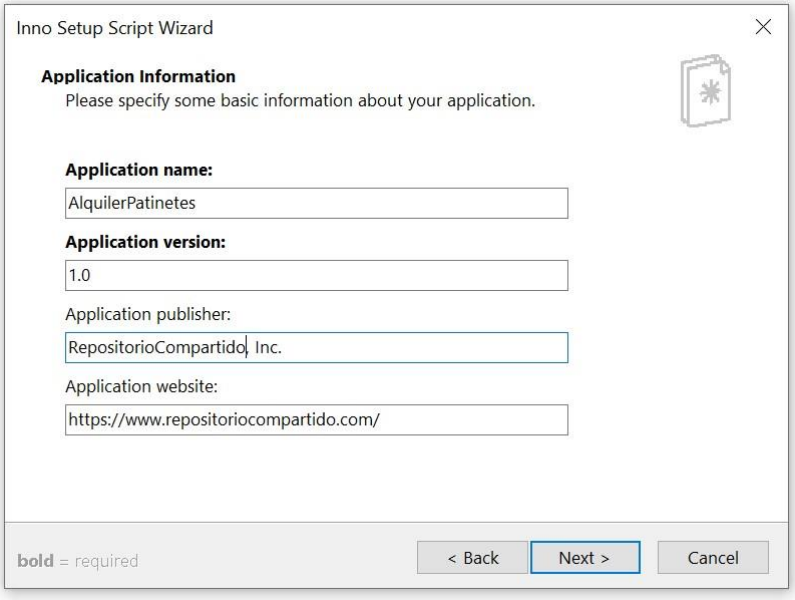
Para la creación del paquete de instalación necesitaremos seguir los siguientes pasos:

1. Creamos un script de instalación usando el Wizard.



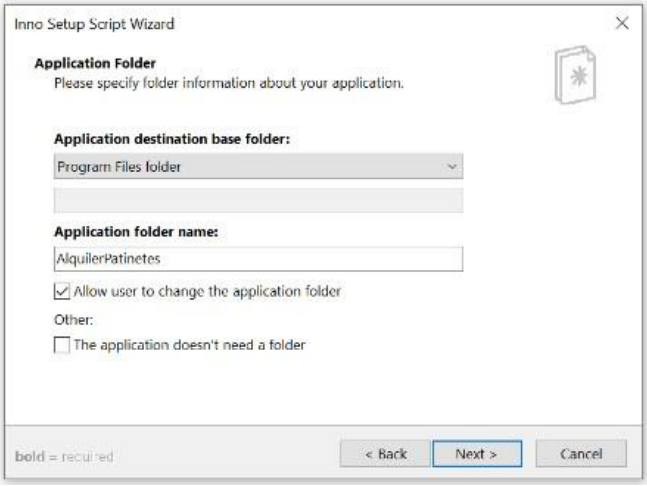
Pantalla inicial Inno Setup
Fuente: Elaboración propia

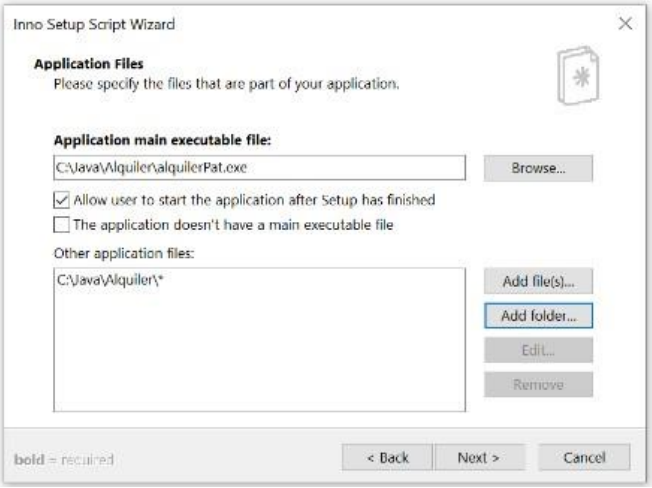
2. Añadimos los parámetros metadata de configuración de la aplicación y el nombre del grupo de instalación que se creará.

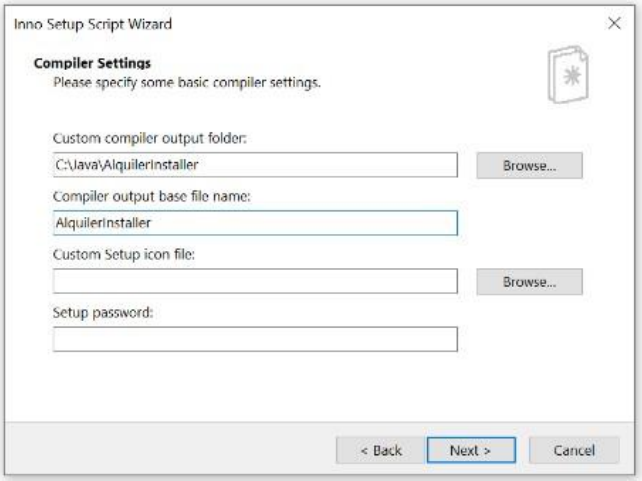


Metadata del instalable
Fuente: Elaboración propia

3. Elegimos el ejecutable previamente creado, por ejemplo, con launch4j y añadimos también todo el contenido del directorio.

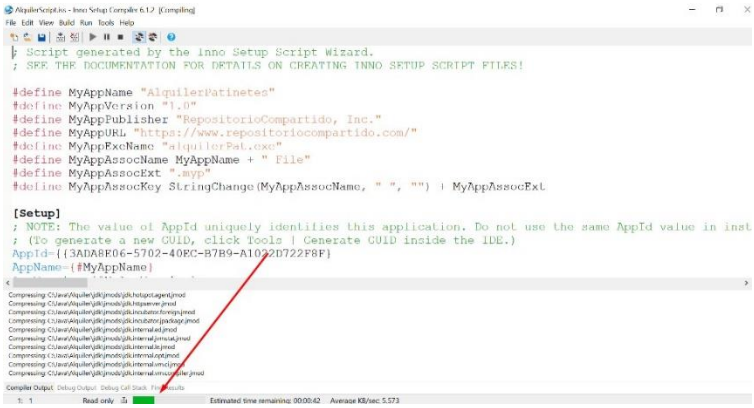






Grupo de aplicaciones destino

Fuente: Elaboración propia



Proceso de creación del instalable

Fuente: Elaboración propia

Otro parámetro a tener en cuenta es el de instalación desatendida. La instalación desatendida es un tipo de instalación que permite ejecutar el instalador sin intervención del usuario. Para activar estos modos, se debe incluir el parámetro correspondiente en el script de instalación (ya que el asistente de Inno Setup no nos muestra esta opción): SilentInstall=yes

```
1  ; -- Example1.iss --
2  ; Demonstrates copying 3 files and creating an icon.
3
4  ; SEE THE DOCUMENTATION FOR DETAILS ON CREATING .ISS SCRIPT FILES!
5
6  [Setup]
7  AppName=My Program
8  AppVersion=1.5
9  WizardStyle=modern
10 DefaultDirName={autopf}\My Program
11 DefaultGroupName=My Program
12 UninstallDisplayIcon={app}\MyProg.exe
13 Compression=lzma2
14 SolidCompression=yes
15 OutputDir=userdocs:Inno Setup Examples Output
16 SilentInstall=yes
17 SilentUninstall=yes
18
19 [Files]
20 Source: "MyProg.exe"; DestDir: "{app}"
21 Source: "MyProg.chm"; DestDir: "{app}"
22 Source: "Readme.txt"; DestDir: "{app}"; Flags: isreadme
23
24 [Icons]
25 Name: "{Group}\My Program"; Filename: "{app}\MyProg.exe"
26
```

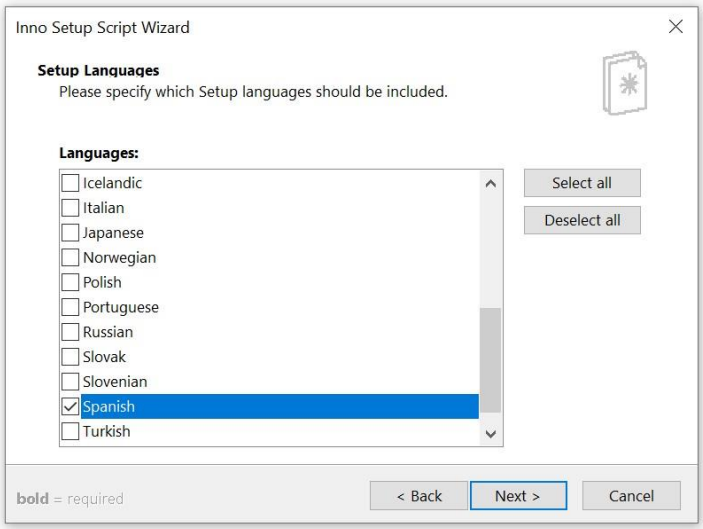
Instalación Silenciosa

Fuente: Elaboración propia

2.2.3 Logotipos, fondos, diálogos, botones, idioma, entre otros

La adaptación de nuestro paquete de instalación a través de Inno Setup es sencilla, ya que en algunos casos se puede realizar directamente a través del mismo menú de creación, y otras a través del script que se configura para la adaptación del paquete

autoinstalable. Uno de los ejemplos lo vemos en la siguiente imagen y la posibilidad de incorporar nuevos idiomas.



Añadir idiomas
Fuente: Elaboración propia

Dentro del script de generación del paquete, como decíamos, tenemos muchas opciones como las que vamos a ver a continuación:

- **Iconos y logotipos**, permite definir la imagen de los mismos.

  [Icons]
  Name: "{group}\My Program"; Filename: "{app}\MYPROG.EXE"; WorkingDir: "{app}"
  Name: "{group}\Uninstall My Program"; Filename: "{uninstallexe}"

- **Colores como el del fondo del instalable.**

  BackColor=clBlue
  BackColor2=clBlack

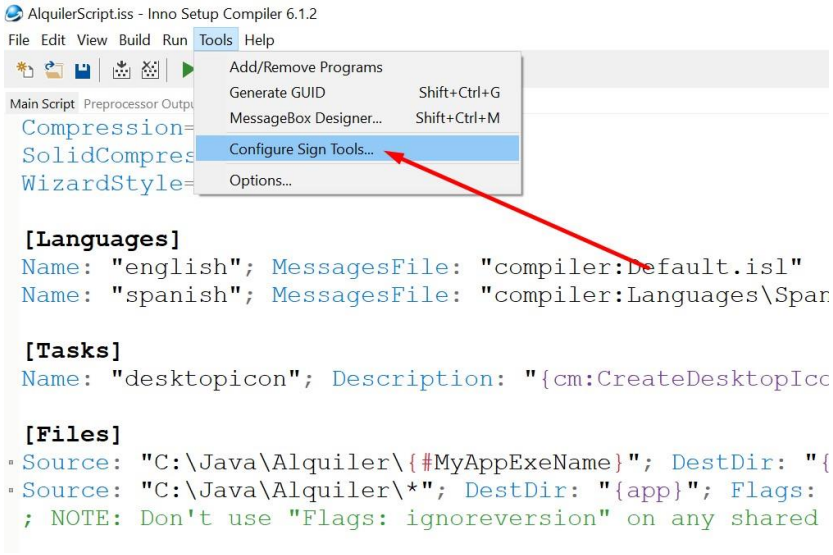
  BackColor=$FF0000
  BackColor2=$000000

- **Mensajes, diálogos y botones**, se pueden definir nuevas secciones a través de las secciones [MEssages].

  [Messages]
  ButtonNext=&Forward >

2.2.4 Ficheros firmados digitalmente

Una vez creado el script base, podemos realizar múltiples configuraciones, como las que veíamos en el anterior apartado, y dentro de estas opciones tenemos la de firmado.



Configuración del script de instalación

Fuente: Elaboración propia

A partir de ahí, podemos usar cualquier herramienta que nos permita añadir un certificado, que firme digitalmente esta instalación o usar un certificado generado por una empresa certificadora.

Windows trae la herramienta signtool cuando instalamos VisualStudio, aunque podemos usar cualquier otra herramienta que nos permita realizar esta tarea.

Herramienta de signtool

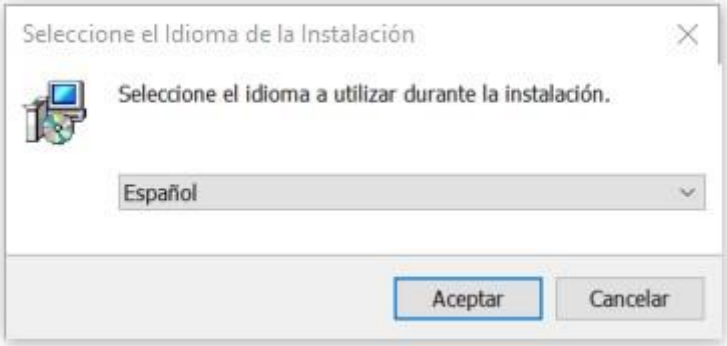
Fuente: Elaboración propia

2.3 Asistentes de instalación y desinstalación

Una vez que hemos generado el instalable, el usuario tendrá un asistente de instalación completo para con el paso a paso poder realizar una instalación completa de la aplicación.

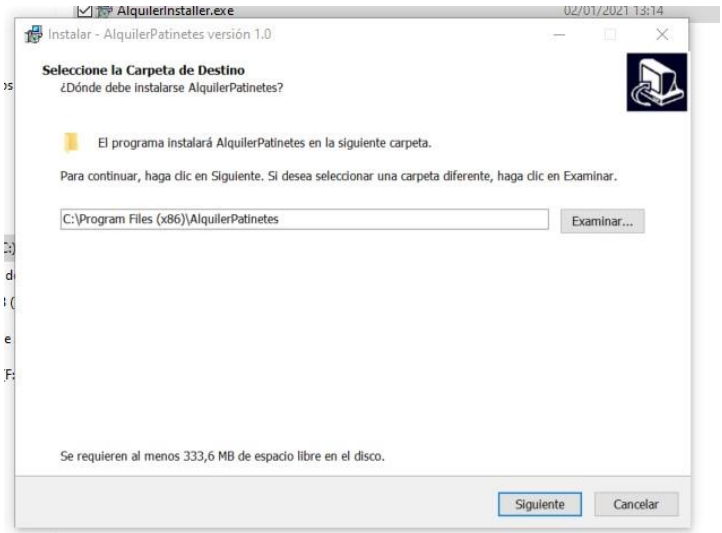
Los pasos habituales en cualquier asistente de instalación son:

1. Selección del idioma de la instalación.



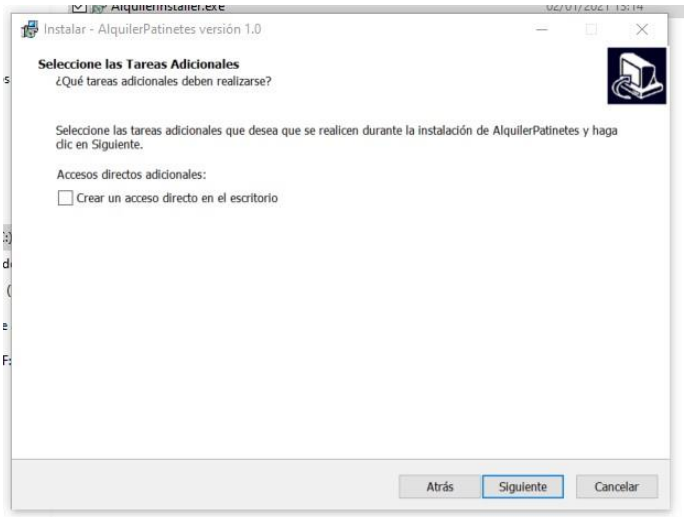
Pantalla inicial de instalación
Fuente: Elaboración propia

2. Selección de la carpeta destino del programa.



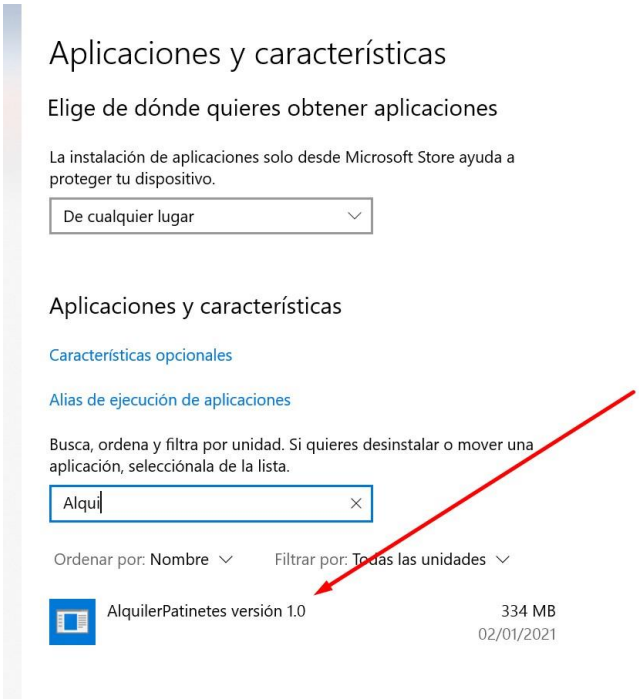
Proceso de instalación
Fuente: Elaboración propia

3. Tareas adicionales y/o ventanas de interacción con el usuario adicionales.



Proceso de instalación
Fuente: Elaboración propia

4. Todos los sistemas operativos se encargan de generar un desinstalador a partir de esa instalación. En el caso de Windows 10, lo tendremos en el menú de Aplicaciones y características.



Aplicaciones y características de Windows
Fuente: Elaboración propia

ENLACE DE INTERÉS

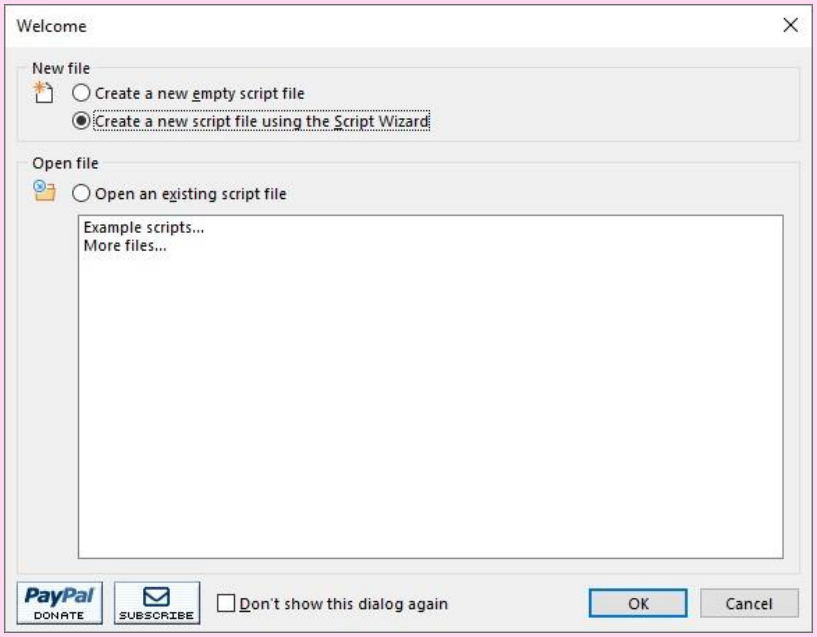
Conoce algunos consejos para la instalación del software de una manera segura:

EJEMPLO PRÁCTICO

En la empresa en la que trabajamos, el equipo de desarrollo ha finalizado una aplicación de escritorio en Java denominada HelloFX y nos proporciona el fichero en formato JAR. Nuestro jefe nos solicita que generemos un instalador con la aplicación Inno Setup.

¿Cómo deberíamos hacerlo?

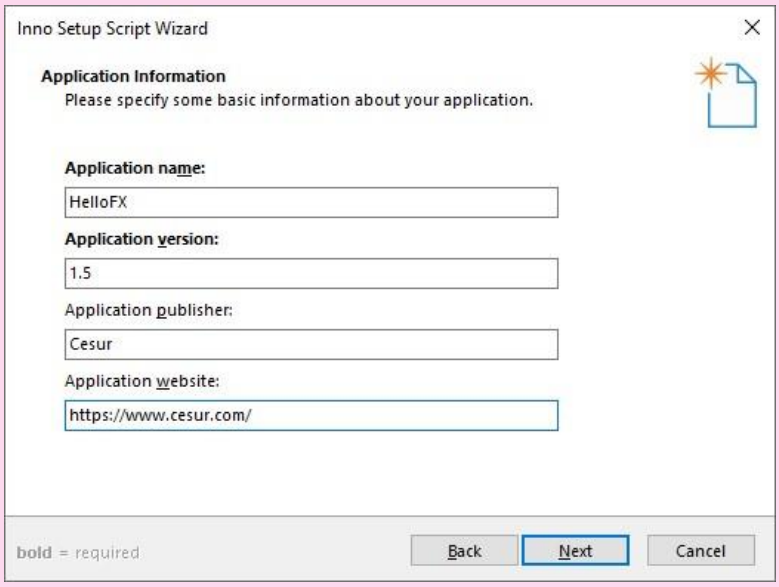
1) Para empezar, ejecutaremos Inno Setup y utilizaremos el Wizard para crear un nuevo script de instalación.



Creación Script

Fuente: Elaboración propia

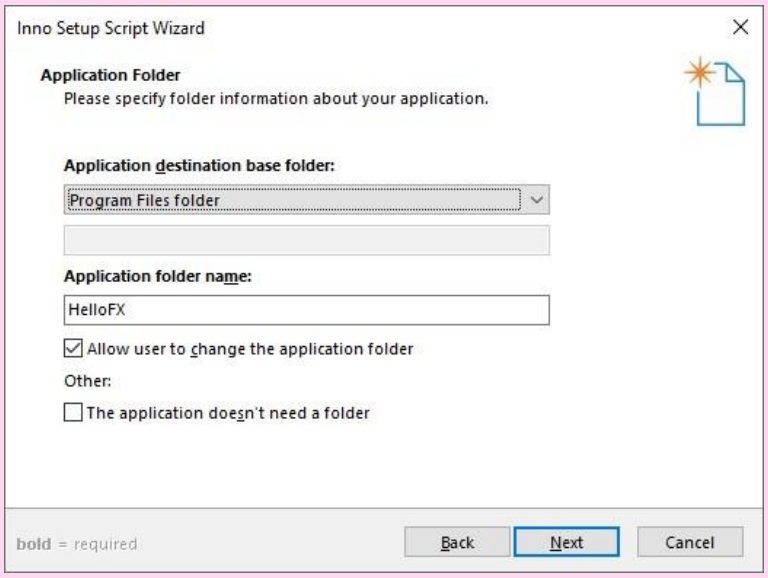
2) Introduciremos el nombre de la aplicación, la versión, el nombre del autor y la URL del sitio web de la empresa.



Datos Instalación

Fuente: Elaboración propia

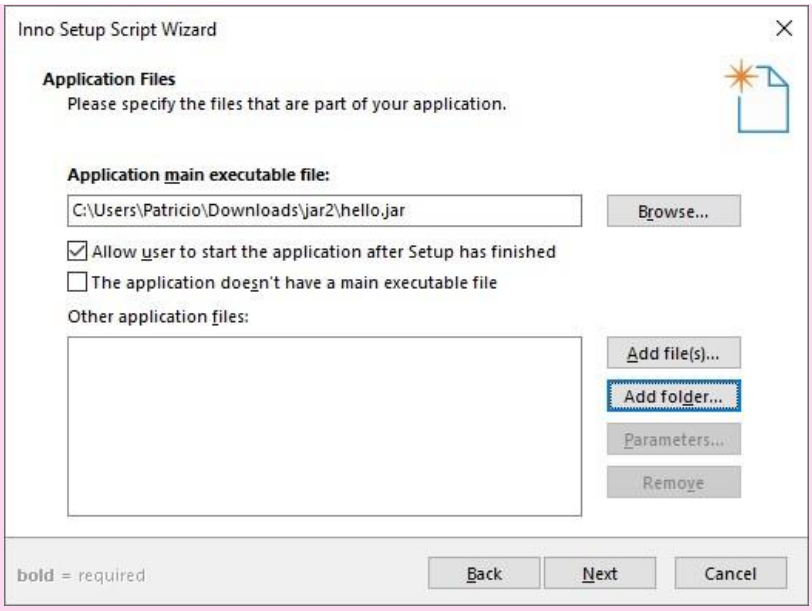
3) Definiremos la carpeta predeterminada, por ejemplo, C:\Program Files\Cesur\HelloFX



Ruta Instalación

Fuente: imagen propia

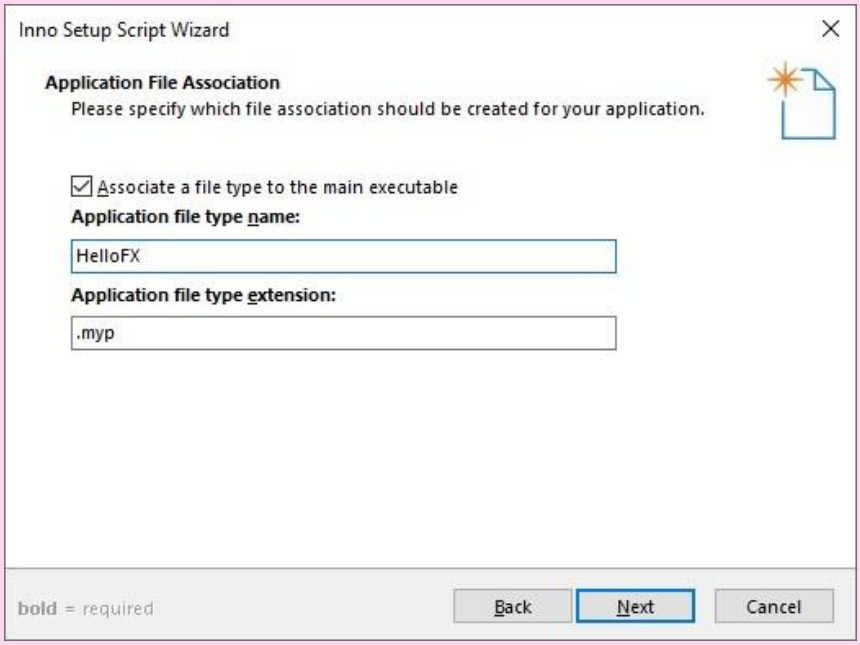
4) Le pasamos el fichero jar para que después de la instalación lo pueda ejecutar, si prevemos que el usuario puede no tener el JRE lo podríamos incluir en la aplicación pulsando en “Add folder..”



Ejecutables Instalación

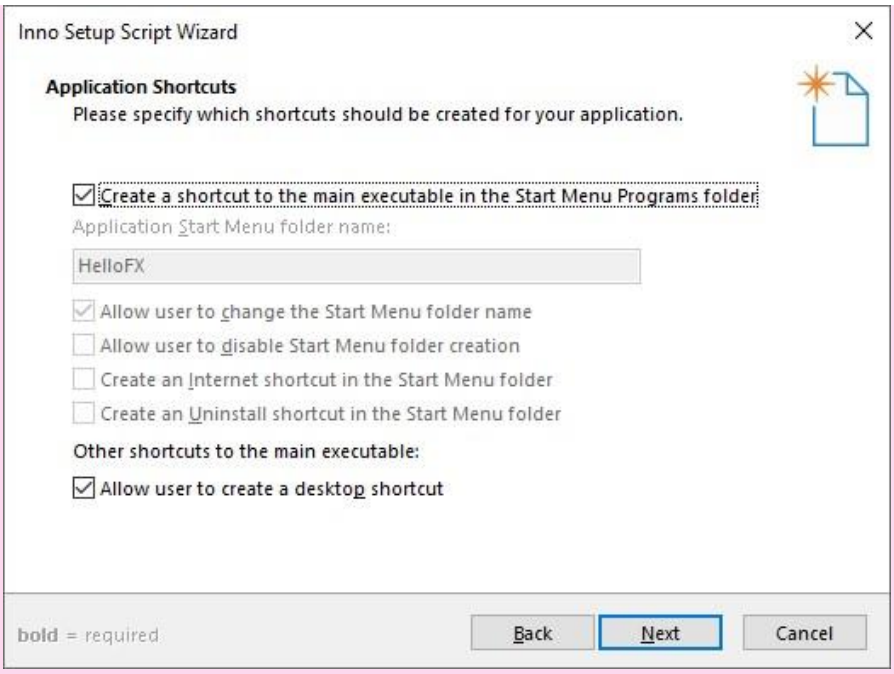
Fuente: Elaboración propia

5) Finalizamos el proceso.



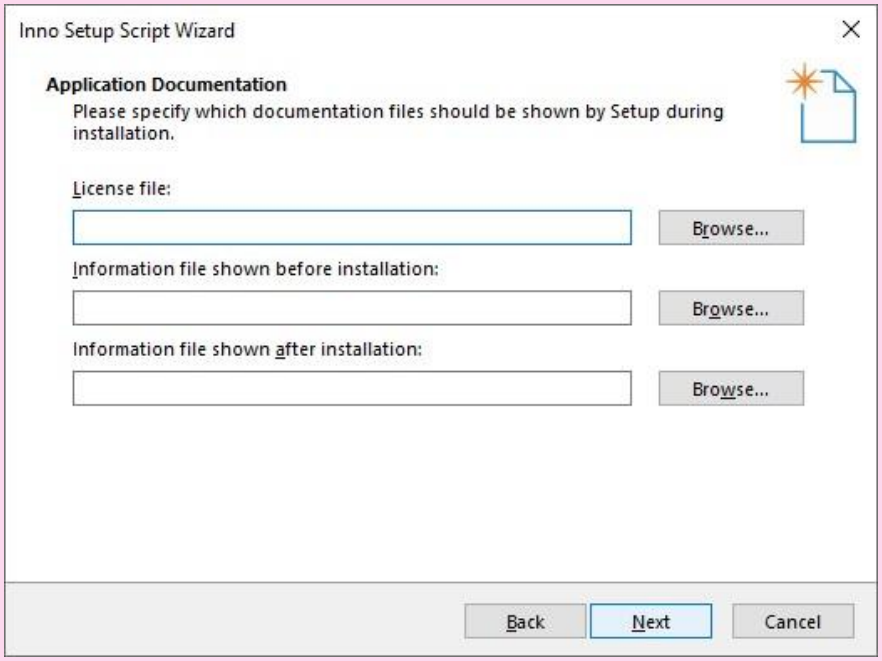
Finalizando Instalación I

Fuente: Elaboración propia



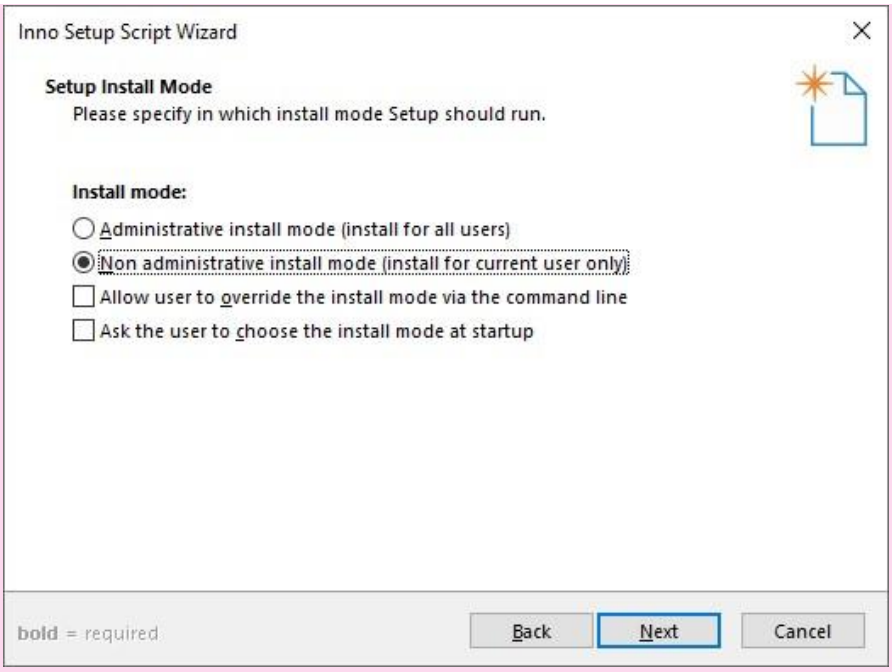
Finalizando Instalación II

Fuente: Elaboración propia



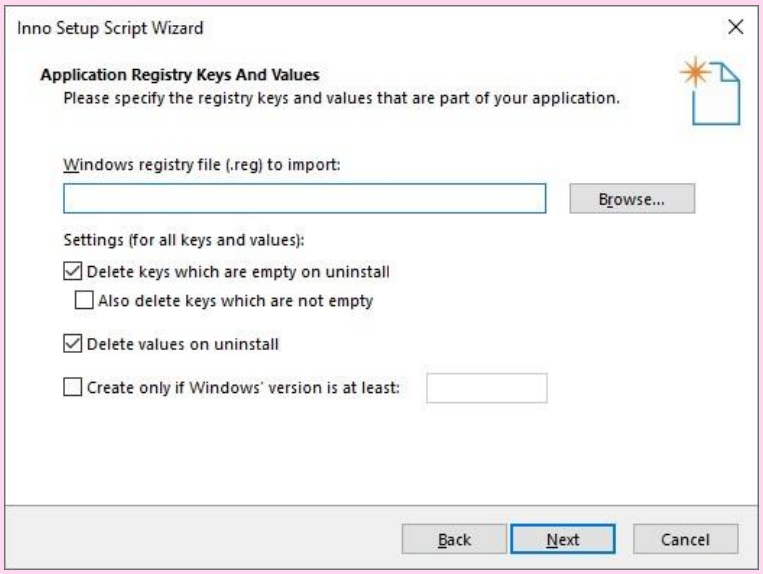
Finalizando Instalación III

Fuente: Elaboración propia



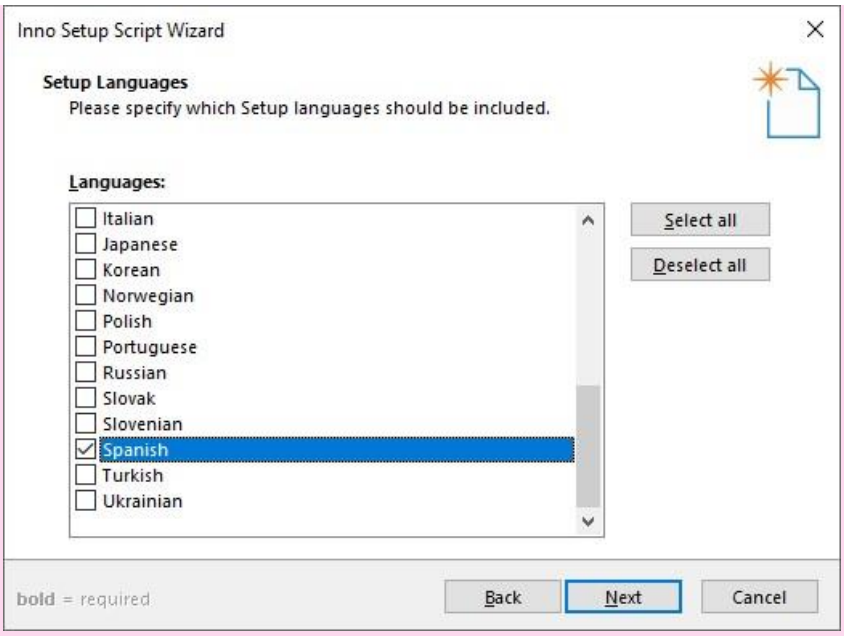
Finalizando Instalación IV

Fuente: Elaboración propia



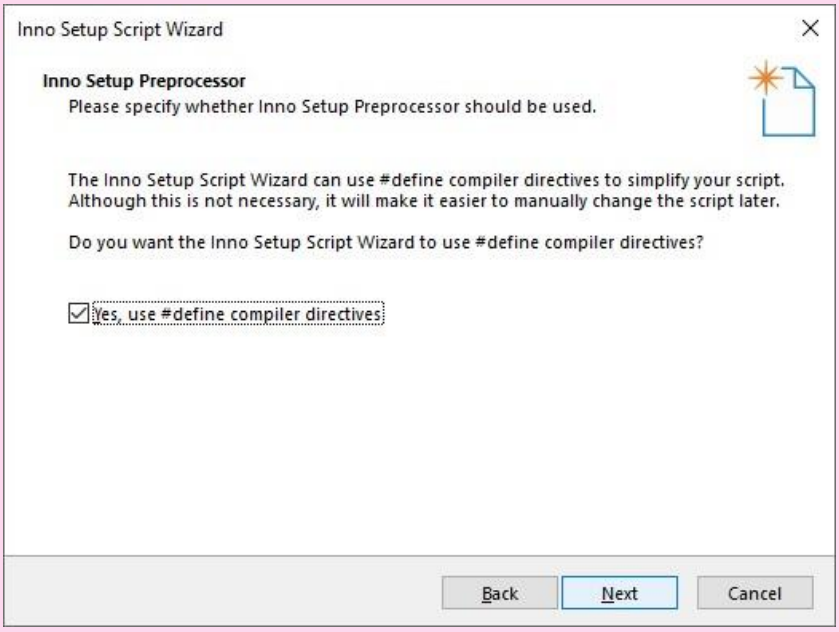
Finalizando Instalación V

Fuente: Elaboración propia



Finalizando Instalación VI

Fuente: Elaboración propia



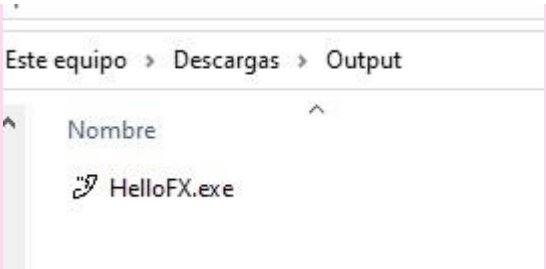
Finalizando Instalación VII

Fuente: Elaboración propia

6) Cambiamos parámetros para dar personalidad (ojo ya que el icono tiene que ser un fichero con extensión .ico en tu ordenador):

OutputBaseFilename=mysetup -> OutputBaseFilename=HelloFX
SetupIconFile= C:\Users\Patricio\Downloads\phone.ico

7) Una vez compilado (Ctrl + F9) el script tendremos el instalador.

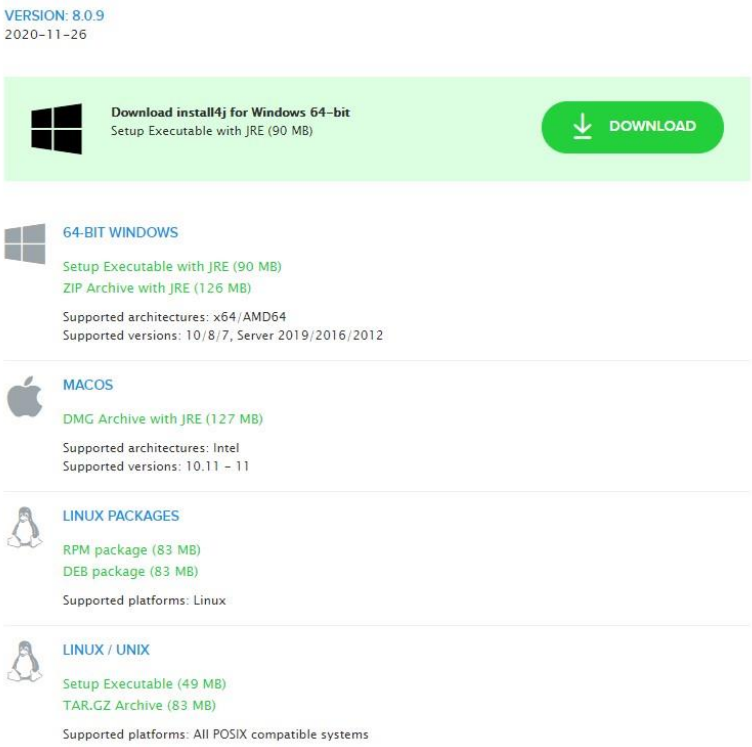


Instalador
Fuente: Elaboración propia

2.4 Instalación, descarga y ejecución desde un servidor web

La distribución de aplicaciones a través de servidores web se ha convertido en una práctica común para la instalación de software tanto para usuarios de sistemas operativos desktop como móviles. Los principales canales de distribución utilizados para la instalación, descarga y ejecución de aplicaciones desde servidores web son:

- **Repositorios (Stores)**: los repositorios o tiendas de aplicaciones son plataformas centralizadas donde los usuarios pueden buscar, descargar e instalar aplicaciones de manera segura y verificada. Estos stores gestionan versiones, actualizaciones y garantizan la compatibilidad con el sistema operativo correspondiente. Tenemos ejemplos de escritorio como Microsoft Store para Windows, Mac App Store para macOS, y GNOME Software o KDE Discover para distribuciones Linux. Para móviles tenemos Google Play Store para Android y App Store para iOS.



Opciones de instalación según el sistema operativo

Fuente: Elaboración propia

Google Play Store Android

Fuente: https://m.europapress.es/portaltic/software/noticia-google-play-prueba-nueva-seccion-unificadas-resenas-20200113182518.html

- **Distribución Ad-Hoc:** consiste en la instalación de aplicaciones directamente a dispositivos específicos sin pasar por una tienda de aplicaciones oficial. Este método es común en entornos empresariales o para pruebas beta.

- **Sitios Web**: consiste en alojar las aplicaciones directamente en sus propios sitios web, ofreciendo descargas directas para los usuarios.

- **Correo Electrónico**: en entornos corporativos o aplicaciones personalizadas, las aplicaciones pueden distribuirse directamente a través de correos electrónicos.

EJEMPLO PRÁCTICO

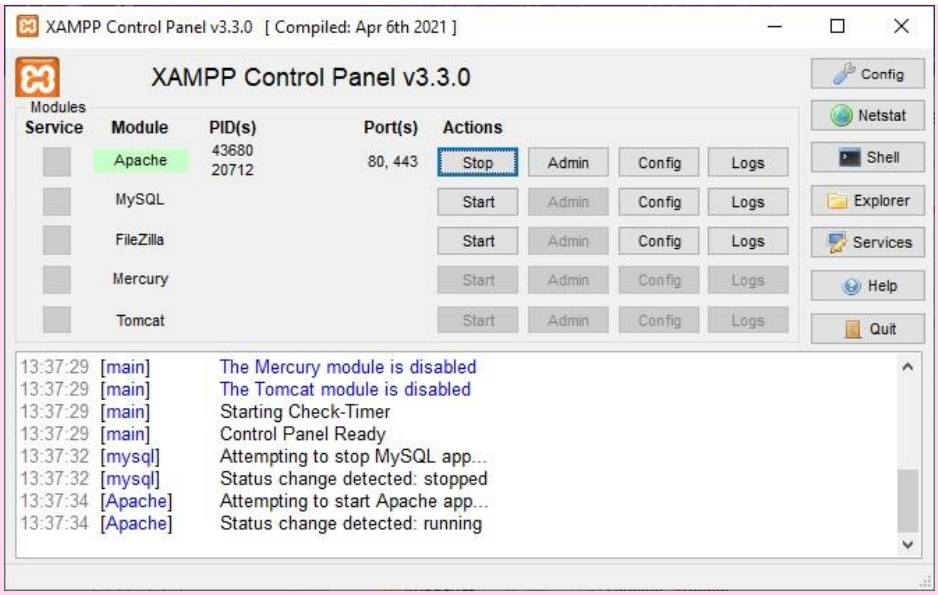
Nuestro jefe solicita nuestra ayuda porque quiere distribuir el instalador de la aplicación a través de un servidor que tenemos ubicado en nuestra empresa. ¿Cómo deberíamos proceder?

1) Instalaremos un servidor de distribución a través de Apache, por ejemplo, XAMPP. Una vez lo instalemos es necesario marcar Apache y PHP.

Instalación Xampp

Fuente: Elaboración propia

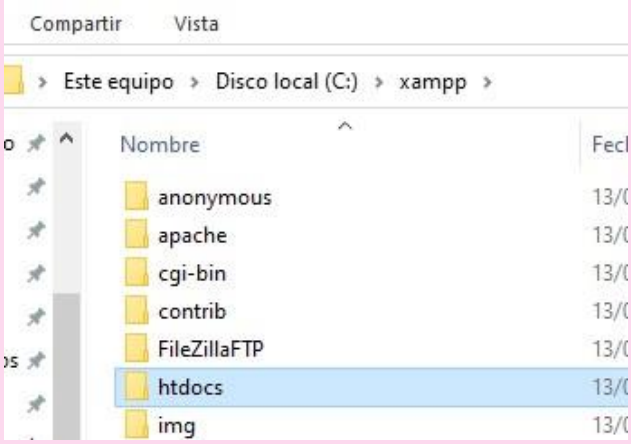
2) En el panel de control de XAMPP, hacemos clic en "Start" para activar el servicio de Apache.



Lanzamiento Apache

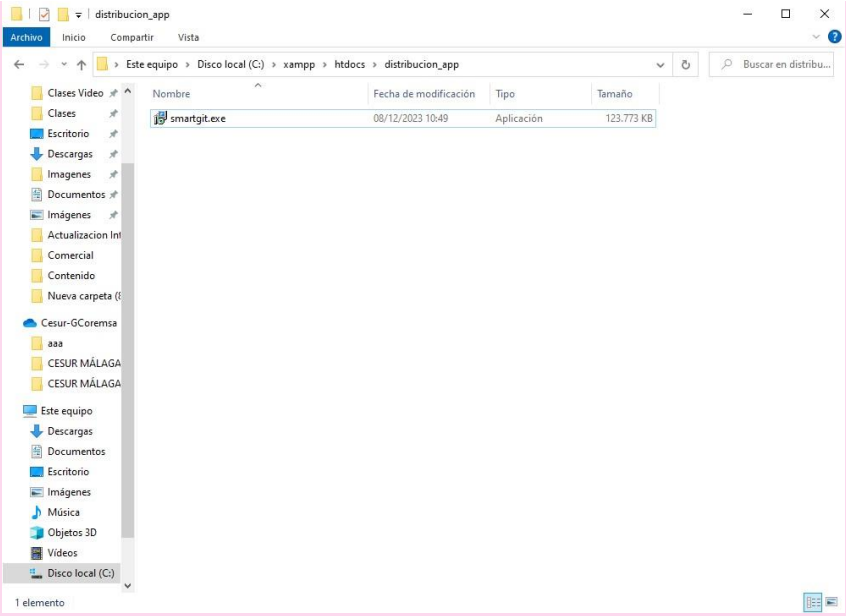
Fuente: Elaboración propia

3) Ahora procederemos a distribuir el instalador. Para ello localizaremos la carpeta htdocs (en Windows C:\xampp\htdocs). Dentro de htdocs, crearemos una nueva carpeta llamada distribucion_app. Y colocaremos en esta carpeta el archivo instalador de la aplicación.



Carpeta htdocs

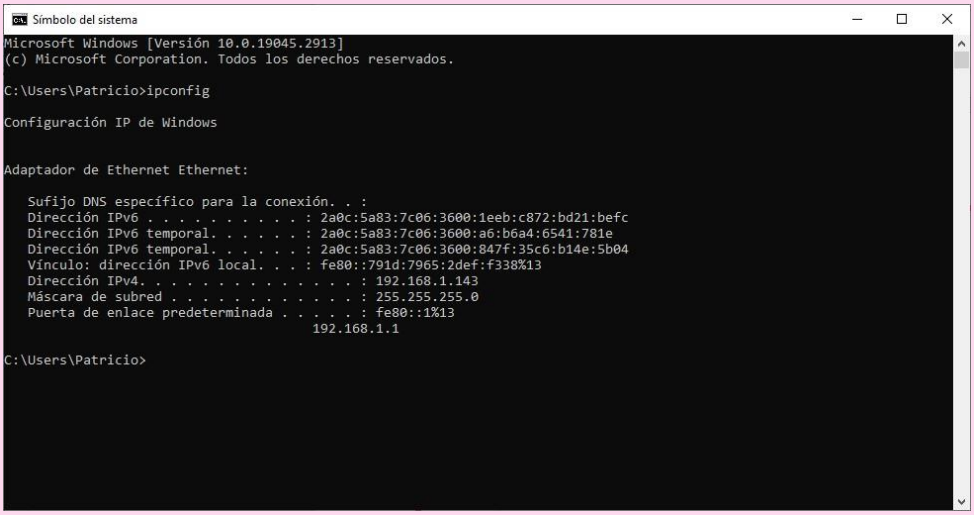
Fuente: Elaboración propia



Instalador Programa

Fuente: Elaboración propia

4) Probamos la accesibilidad del instalador. Para ello identificamos la IP privada del servidor con ipconfig, en este caso 192.168.1.143. Introducimos en el navegador http://192.168.1.143/distribucion_app/ y comprobamos si la aplicación es descargable.

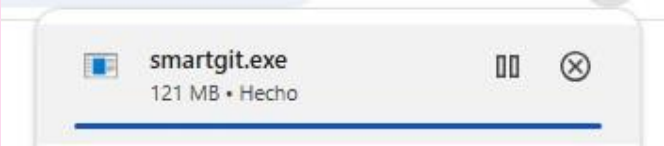


Identificación de IP

Fuente: Elaboración propia

Accesibilidad del Servidor

Fuente: Elaboración propia



Descarga del instalador distribuido

Fuente: Elaboración propia

RESUMEN FINAL

Dentro de las fases del desarrollo de aplicaciones, la instalación, despliegue y mantenimiento de una aplicación es esencial, ya que de esa forma nuestras aplicaciones son usadas y puestas en funcionamiento.

Tal y como hemos visto, la dependencia con el sistema operativo final es total, ya que el software que vamos a instalar va a ser utilizado por un usuario normalmente en un sistema operativo.

Si realizamos nuestros desarrollos mediante Java, el formato JAR nos permite empaquetar nuestra aplicación y ser usada en múltiples sistemas operativos, ya que será traducida y compilada por el JRE de Java previamente instalada en el equipo.

También hemos introducido cómo realizar instalables dependientes del sistema operativo, ya que el usuario final, que normalmente no tiene conocimientos técnicos de software puede de una forma sencilla usar estos programas. Y para mejorar incluso más esta instalación, creamos paquetes autoinstalables que permiten empaquetar e instalar las librerías necesarias para ejecutar el software en el equipo final.